

№1
05.2026

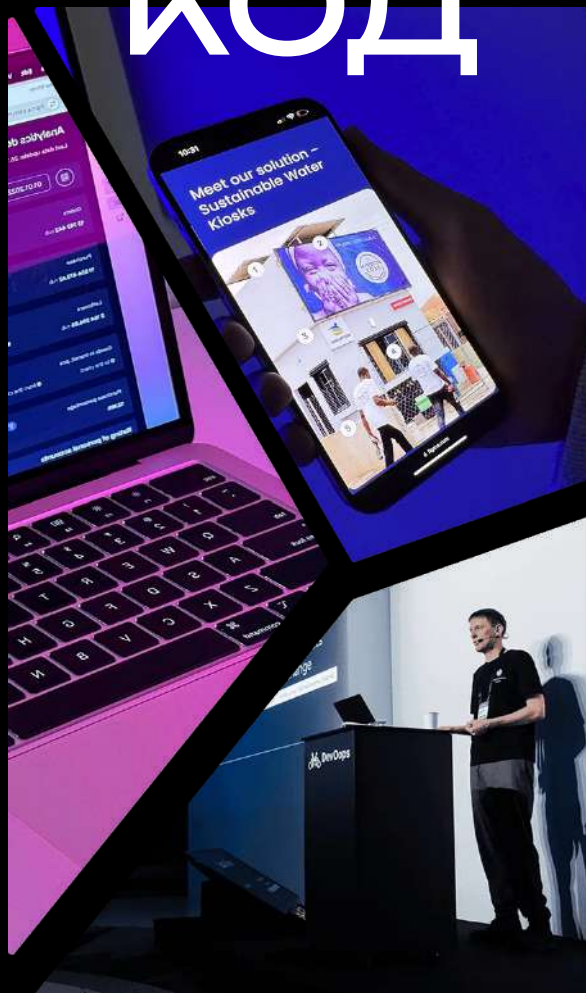
TEST-DRIVEN
DEVELOPMENT

РАЗРАБОТКА ЧЕРЕЗ
ТЕСТИРОВАНИЕ



ТУРМАЛИН КОД

ПРОЕКТЫ
ПО TDD
С ОТКРЫТЫМ
ИСХОДНЫМ
КОДОМ



УСЛОВИЯ,
ПРИ КОТОРЫХ
TDD РАБОТАЕТ

// Инженерный журнал о коде,
процессах и бизнес-решениях

Test-Driven Development (разработка через тестирование)

Привет!

Мы — Tourmaline Core.

7 лет мы делаем ИТ для очень разных компаний и организаций.

Carl Zeiss и Челябинский зоопарк, медицинские стартапы и австралийские благотворители. Драйверы, ML-модели, веб-системы, прошивки, блокчейн...

За это время накопилось много историй: про то, как принимались решения, что из этого получалось, где мы ошибались и какие выводы помогли делать следующие проекты иначе. Так шпала за шпалой прокладывались рельсы, на которые мы ставим новые проекты.

Часть из наших историй становится статьями на Хабре. Часть — докладами на конференциях. Но какие-то кусочки пазла не помещаются в эти форматы: это истории, у которых много участников, много отсылок, и в которых одна тема цепляется за десяток других. Из таких историй и будет собираться этот журнал.

Первый выпуск про Test-Driven Development. Тему мы выбрали не случайно: несколько лет назад наш руководитель принял принципиальное решение — дальше работаем по TDD везде, где это возможно. Сейчас это основа нашего инженерного фокуса.

И на каждом проекте это решение мы объясняем как техническим специалистам со стороны заказчика, так и управленцам. Решили, что пора собрать наш ответ обеим сторонам в одном месте.

Мария Ядрышникова,
Директор по развитию



В 2001 году журнал «Хакер» опубликовал легендарный материал «Если бы программисты строили дома». Бригада забыла фундамент и построила первый этаж, пришлось все ломать. Потом на пятом этаже стена ушла под 40 градусов. Ещё неправильно положили какой-то кирпич, а какой именно — непонятно, проще снести и переделать. Крыша строилась отдельно и не подошла к дому. Лифт после оптимизации пробил крышу и улетел.

Конечно, перед стройкой настоящего дома всё проектируют до старта. Договариваются обо всём: сколько этажей, какая нагрузка на перекрытия, где несущие стены, какую делать отделку и из чего. Техническое задание — это формулировка того, каким должен получиться дом.

В разработке ПО часто делают наоборот. Программист пишет код, а потом пишет автоматизированные тесты к тому, что уже получилось. И то не всегда. Эти тесты подтверждают принятые в процессе разработки решения, а не помогают их проектировать.

Поэтому часто ИТ-системы — это дом из байки «Хакера».

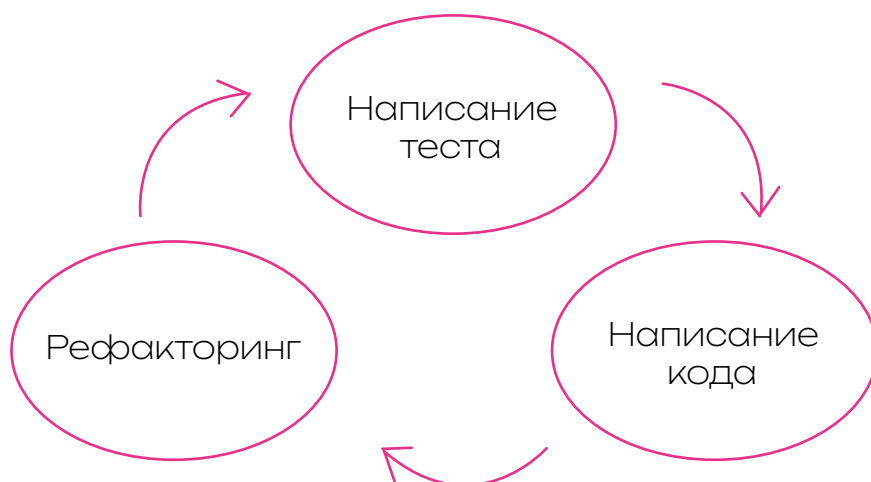
TDD (Test-Driven Development)

— это разработка, в которой сначала договариваются, потом делают.

Программист записывает, как новая функция должна себя вести: что подаём на вход, что должны получить, что считается ошибкой. Эта запись называется тестом, она проверяет код автоматически. Потом пишется код реализации. После этого код приводят в порядок, улучшают вид, не меняя функционал — это называется рефакторингом.

Цикл повторяется десятки раз в день, маленькими шагами.

И почему же мы выбрали такой путь: работать по TDD? Попробуем показать на примере комикса.



Источник



Путешествие во времени: цена устойчивости

разработчик



клиент



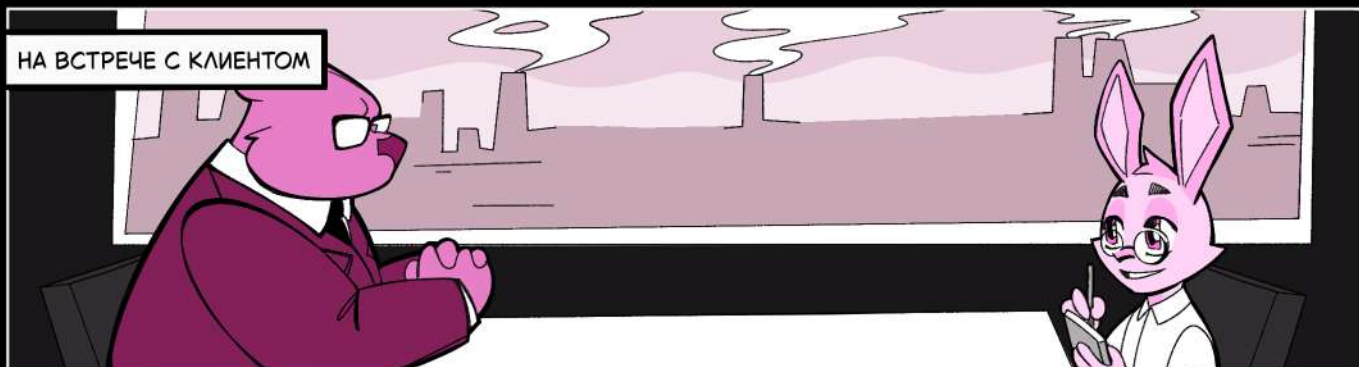
менеджер



менеджер



НА ВСТРЕЧЕ С КЛИЕНТОМ



У НАС МНОГО ПОДРЯДЧИКОВ.
ПОЧЕМУ МЫ ДОЛЖНЫ
ВЫБРАТЬ ВАС?



НУ, ЕСЛИ
КРАТКО...

ОЧЕНЬ МНОГО ГОВОРИТ О
МНОГО ГОВОРИТ ОЧЕНЬ МНО
ГОВОРИТ ОЧЕНЬ МНОГО ГОВО
ОЧЕНЬ МНОГО ГОВОРИТ ОЧЕН
МНОГО ГОВОРИТ ОЧЕНЬ МНО
ГОВОРИТ ОЧЕНЬ МНОГО ГОВО
ОЧЕНЬ МНОГО ГОВОРИТ ОЧЕН
МНОГО ГОВОРИТ ОЧЕНЬ МНО
ГОВОРИТ ОЧЕНЬ МНОГО ГОВ
ОЧЕНЬ МНОГО ГОВОРИТ ОЧЕН
МНОГО ГОВОРИТ ОЧЕНЬ МНО
ОЧЕНЬ МНОГО ГОВОРИТ ОЧЕН



НАШЕЙ ИТ-СИСТЕМЕ
УЖЕ 12 ЛЕТ.
СДЕЛАТЬ ДЛЯ НЕЕ
МОДУЛЬ – БОЛЬШАЯ
ОТВЕТСТВЕННОСТЬ

ВАУ КАК КРУТО!
ХОЧУ ПРИЛОЖИТЬ
К ЭТОМУ РУКУ!

ОБСУДИМ С КОМАНДОЙ,
ЧТО МЫ МОЖЕМ
ВАМ ПРЕДЛОЖИТЬ,
И ВЕРНУСЬ



ПРИДУМАЛ, КАК
ВПЕЧАТАЛИТЬ
ЗАКАЗЧИКА!



В ОФИСЕ

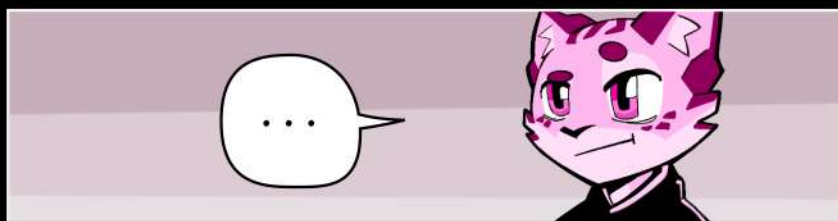
-20%
БЮДЖЕТА



КЛИЕНТ НЕ УВИДИТ ТЕСТЫ,
ЗАТО УВИДИТ ЭКОНОМИЮ.
УБИРАЕМ TDD, ОСТАВЛЯЕМ РЕЗУЛЬТАТ



...





ПРОХОДИТ 1 МЕСЯЦ.
СОЗВОН С ЗАКАЗЧИКОМ

КАК ХОРОШО, ЧТО
УСПЕЛИ ВОВРЕМЯ!

ПЕРЕДАЕМ НА ТЕСТИРОВАНИЕ
НАШЕМУ ОТДЕЛУ КАЧЕСТВА

НУ ВОТ!
ВСЕ ЖЕ ХОРОШО

ПРОСТО
СМОТРИ
ДАЛЬШЕ...

ПРОХОДИТ МЕСЯЦ

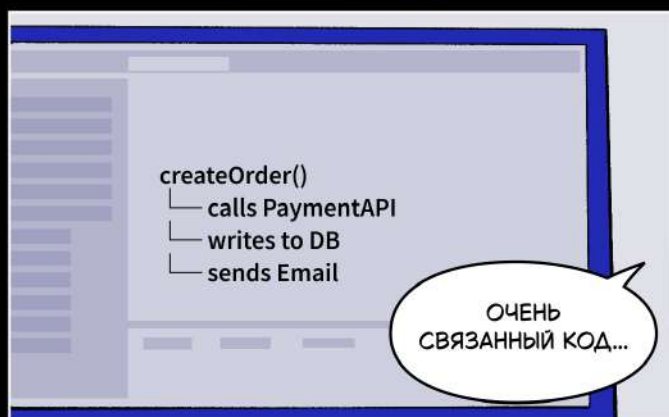
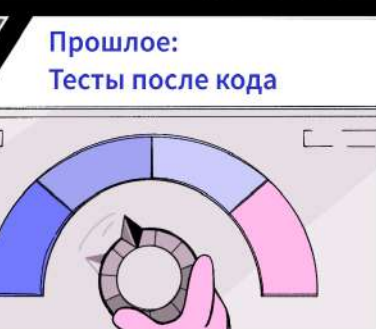
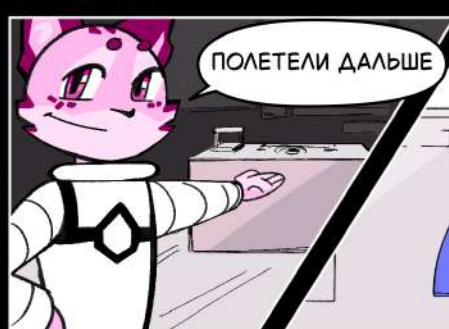
КЛИЕНТ:
ТАМ ЕЩЕ ОДИН БАГ НАШЛИ!

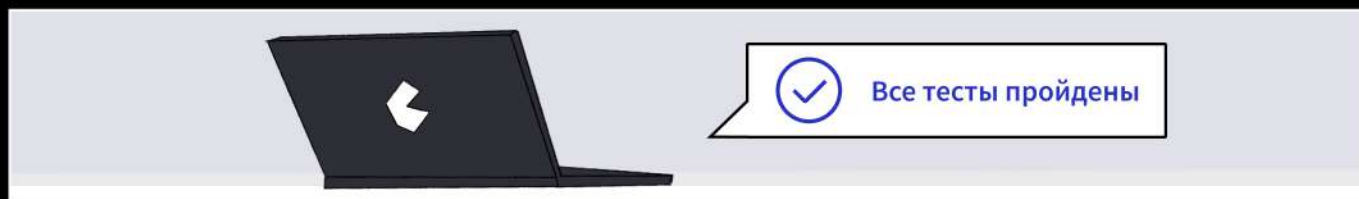
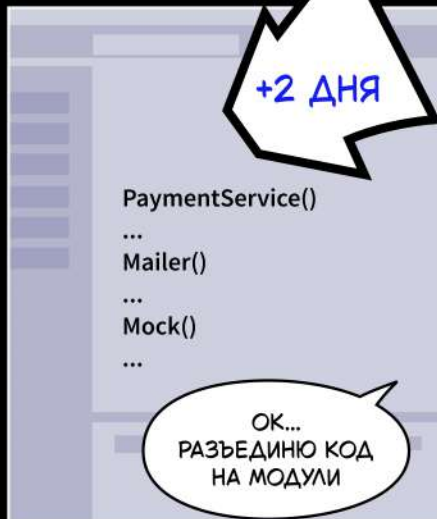
КЛИЕНТ:
ВЫ ПРЕДЫДУЩИЕ ЕЩЕ НЕ ЗАКРЫЛИ!!

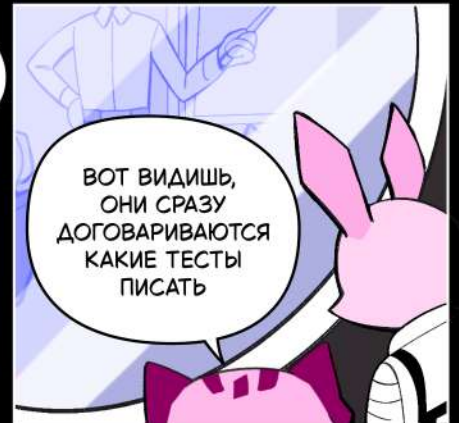
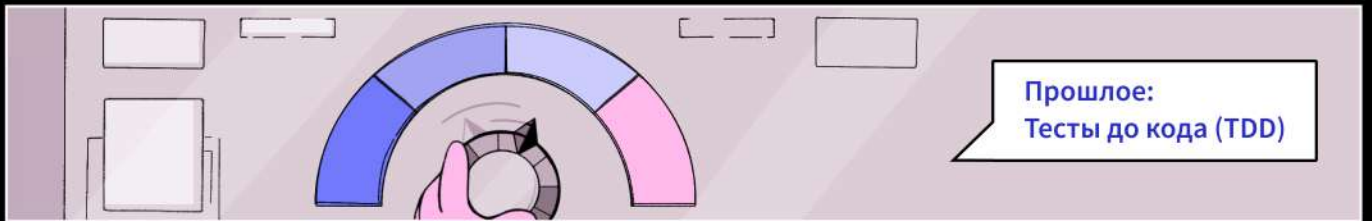
КЛИЕНТ:
КАК ВЫ МОГЛИ ТАК НАЛАЖАТЬ!

ПОТОМ ЕЩЕ МЕСЯЦ
РАЗГРЕБАЛИ.
И К РЕЛИЗУ НЕ УСПЕЛИ

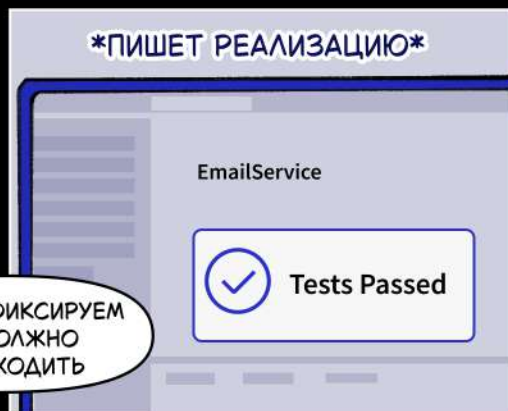
И КЛИЕНТА ПОТЕРЯЛИ
— БОЛЬШЕ ОН НАМ ПРОЕКТ
НЕ ДАСТ



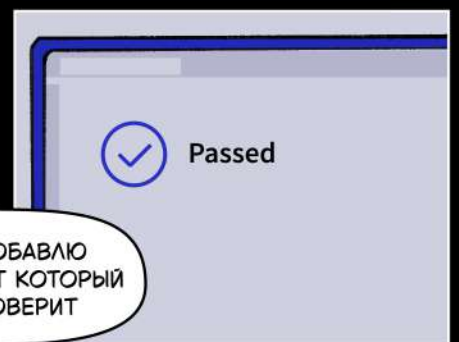




СНАЧАЛА ФИКСИРУЕМ ЧТО ДОЛЖНО ПРОИСХОДИТЬ



ВИДИШЬ? ОН СРАЗУ ЗАДАЕТ ГРАНИЦЫ МЕЖДУ ЧАСТЯМИ СИСТЕМЫ. ТЕСТ ЗАСТАВЛЯЕТ ЕГО ПРОДУМЫВАТЬ ДИЗАЙН КОДА СРАЗУ, А НЕ ПОТОМ





ВСЕ ЗАВИСИТ ОТ ЗАДАЧ
БИЗНЕСА И СИСТЕМЫ,
С КОТОРОЙ БУДЕТ
РАБОТАТЬ РАЗРАБОТЧИК

НЕ СУЩЕСТВУЕТ
УНИВЕРСАЛЬНЫХ РЕШЕНИЙ.
ЕСТЬ ПОДХОДЯЩИЕ
ПОД ТЕКУЩУЮ ЗАДАЧУ



Подход	Что получается	Когда оправдано
Без тестов	Быстро на старте, дорого потом	MVP. Прототип. R&D-эксперимент. Весь код, который готовы выкинуть и начать писать заново
Тесты после кода	Контроль есть, но дорого менять	Система с горизонтом 1—1,5 года. Стабильные требования
TDD	Дороже сначала, дешевле на длинном горизонте	Системы с горизонтом от 2-3 лет. Активно меняющиеся требования. Высокая цена бага в проде

Ручное тестирование как отдельную полноценную стратегию в таблицу мы не выносим. Оно не масштабируется на проекты с длинным горизонтом и частыми изменениями, и в большинстве случаев заменяется автотестами или точно дополняет их.

Когда мы говорим заказчику «давайте работать по TDD», реакция предсказуема: звучит конечно, красиво, но это же дольше и дороже. Да и как оценить эффект? Кто сказал, что будет качественнее?

В 2008 году исследователи из Microsoft Research, IBM и Университета Северной Каролины опубликовали результаты по четырём промышленным командам. Три из них из Microsoft: это команды, разрабатывающие Windows, MSN и Visual Studio. Одна команда из IBM, которая разрабатывала драйверы для устройств.

Команды и продукты совершенно разные: разный объем кода (от 6 до 155 тысяч строк кода), разное время эксперимента (от 24 до 119 человеко-месяцев), разная доменная область проектов.

Эти команды работали по TDD. Рядом с ними работали команды с похожими проектами, но без применения TDD-подхода. И вот результат:

Плотность дефектов у TDD-команд оказалась на 40–90% ниже, чем у сопоставимых команд без TDD. За это пришлось заплатить временем: руководители оценили рост сроков начальной разработки в 15–35%.

Важная деталь в том, что оценивались сроки начальной разработки. В этот срок не заложено дальнейшее сопровождение продукта. А именно там и утекает основное время: правки по багам, регрессионное тестирование продукта после любых изменений, разбор инцидентов, погружение новых участников проекта в кодовую базу.

У TDD-команд процессы сопровождения быстрее, потому что дефектов в коде меньше, тесты документируют поведение, а менять покрытый автотестами код можно без страха: автотесты покажут, если что-то сломалось.

То есть 15–35% времени и бюджета — это цена, которую нужно заплатить один раз, в обмен на качество, которое работает дальше.

Рентабельность использования TDD-подхода можно оценить по времени жизни самого проекта: если проект будет жить долго, то вложения окупятся.

Источник



Противоречащее исследование

Biz

В 2021 году вышел метаанализ из 12 экспериментов. В этом эксперименте участвовали новички в программировании и опытные программисты, но ни у кого из них не было опыта работы с TDD. Задачи были учебными, и на них отводилось 2.5 часа. В этом исследовании TDD не показал преимущества над итеративной разработкой с короткими циклами с тестами после кода.

Это исследование не противоречит исследованию Microsoft и IBM, а добавляет к нему мысль: на дистанции в часы для людей без опыта с TDD эффект не успевает проявиться. Авторы также пишут, что нужны эксперименты с экспертами методологии на длинных задачах. Мы работаем чаще с длинным горизонтом, поэтому ориентируемся на Microsoft и IBM.

Источник



Biz

Семь условий, при которых TDD начинает работать

В 2011 году исследователи из Mälardalen University прошли обзором по работам про TDD в индустрии и задались вопросом: почему практика, которая в исследованиях даёт качество, в работе приживается так плохо? Обычно эту работу цитируют как «семь причин работать без TDD».

Мы переделали выводы из статьи в чек-лист из семи условий, при которых TDD начинает давать тот эффект, который зафиксировали Microsoft и IBM, и который мы можем подтвердить в своих командах.

1

Готовность платить за старт. Команда, руководители и заказчики заранее принимают, что в переходный к TDD-разработке период задачи будут занимать больше времени. Если между всеми участниками процесса нет согласия по этому пункту, под давлением сроков от TDD быстро откажутся.

2

Опыт работы по TDD у команды. Часто проблемы внедрения появляются от непонимания того, как это должно происходить. Начать закрывать эту базу можно с помощью книг основоположника TDD Кента Бека, обучающих видео и open source репозиториях.

Книга
Кента Бека



-
- 3 Навык писать тесты.** Плохо написанные тесты — это тесты хрупкие, дублирующие друг друга, проверяющие реализацию вместо поведения. С плохими тестами TDD превращается в дорогую, сложно поддерживаемую иллюзию контроля.
-
- 4 Минимальный дизайн.** Даже при разработке по TDD, где тесты в том числе помогают проектировать дизайн кода, необходимо перед стартом представлять контуры разрабатываемого модуля: границы, интерфейсы, ответственность. Без этого тесты переписываются на каждом шагу.
-
- 5 Дисциплина соблюдения цикла.** TDD работает только тогда, когда команда действительно придерживается этого подхода, а не имитирует его. На практике дисциплина в этом вопросе может «хромать» из-за нехватки времени или нежелания следовать подходу.
-
- 6 Подходящая предметная область и инструменты.** Авторы статьи пишут про то, что в некоторых областях, например, тестировании интерфейсов, не хватает инструментов и необходимо их развитие. Эта статья 2011 года. С тех пор прошло много времени, и индустрия сделала скачок — для веб-интерфейсов появились, как минимум, Cypress и Playwright. Сейчас индустрия тестирования сильно повзрослела: инструменты можно найти и адаптировать в зависимости от предметной области.
-
- 7 Не легаси.** На непокрытом унаследованном коде TDD в чистом виде не запускается. Сначала нужно отдельным усилием провести рефакторинг системы под новый подход, и потом запускать TDD в бой. Это самостоятельный проект, который стоит оценивать отдельно.
-

Хорошая новость в том, что ни одно из семи условий не упирается в магию или таланты «звёздной команды».

- Готовность платить за старт — вопрос договорённости с заказчиком.
- Опыт, навык писать тесты, дисциплина работы по TDD тренируются.
- Минимальный дизайн и подходящие инструменты закладываются на этапе оценки.
- Легаси выносится в отдельный контур работ.

Каждое из условий — это инженерная или управленческая задача, у которой есть понятное решение.

Источник



Инфраструктура под TDD: что мы стандартизировали, прежде чем взяться за тесты

Tech

С тезисом про подходящие инструменты мы согласны. Но дело не только в том, чтобы выбрать правильную библиотеку и завтра начать. Перед этим должна быть подготовлена инфраструктура, выбраны инструменты под конкретный стек и проект, стандартизирована разработка и продана команде сама идея работы по TDD. Test-Driven Development в Tourmaline Core начался с фронтенда, а потом уже перетек в бекенд и эмбедд. По фронтенду у нас

больше всего открытых кейсов и публичных материалов, поэтому эволюцию подхода покажем на его примере. А для бекендеров и эмбеддеров — дополнительные материалы в конце выпуска.

Материал далее написан на основе доклада «TDD на фронтенде» с конференции TechTrain 2024.

Доклад



1

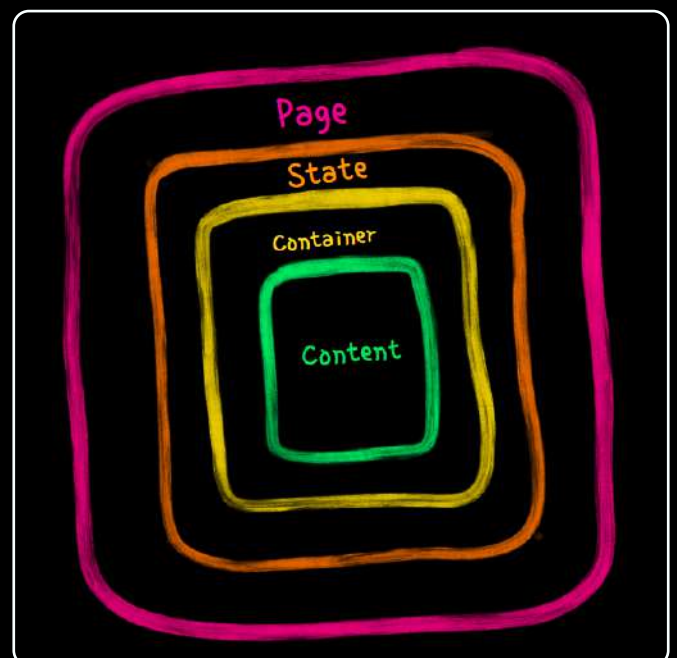
Подход Contract-First

Прежде чем браться за реализацию новой фичи, договариваемся с бекендом о контракте и фиксируем его в Swagger нашего API. Из него генерируются TypeScript-типы: фронтенд сразу получает типизированную модель. Это позволяет разрабатывать его в песочнице с полной уверенностью в том, что в итоге фронт с беком соединятся как надо.

2

Архитектура из четырёх слоёв.

Каждая страница раскладывается на одни и те же уровни: **Page** оркестрирует, **State** хранит данные и бизнес-логику, **Container** отвечает за сетевые вызовы, **Content** — за рендер и интерактив. Каждый слой не налезает на чужую зону ответственности: State ничего не знает про сеть, а Container ничего не рендерит. Эта нарезка одинакова на всех проектах — разработчик с одного проекта понимает структуру другого без онбординга.





Александр, руководитель Tourmaline Core. Выступление на DevOps.

3

State как обычный ES6-класс с Mob X.

Состояние и бизнес-логика лежат в обычных классах, превращённых в наблюдаемые через `makeAutoObservable` — это утилита Mob X. State тестируется как обычный объект: создал, дёрнул метод, проверил состояние.

4

Тестовая стратегия.

Плавно вытекает из архитектуры. Явно зафиксировано, на каком уровне какие тесты писать.

- State-тесты — сложная логика стейта: расчеты, валидации, фильтрации.
- Component-тесты — поведение UI: отключена ли кнопка, вызовется ли коллбек.
- Container-тесты — сетевые сценарии.
- E2E — бизнес-флоу.

Один и тот же код не покрывается с двух сторон, тесты не дублируются. И команда пишет тесты осмысленно и единообразно.

5

Ставка на E2E как тесты с самым высоким ROI.

На фронте самая дорогая поломка — это та, что мешает пользователю пройти ключевой бизнес-сценарий. E2E-тесты, которые покрывают ключевые пользовательские сценарии, ловят такие поломки лучше всего.

TDD на фронте стал возможен не столько из-за хороших библиотек, сколько из-за сведения логики к воспроизводимой структуре: одинаковая нарезка слоёв, одинаковая тестовая стратегия, одинаковый способ работы с контрактом. И конечно, похожий подход можно реализовать и в других направлениях разработки.

Полностью открытый кейс компании — сайт Челябинского зоопарка

Tech

Когда мы говорим, что работаем по TDD, заказчики просят показать примеры. На коммерческих проектах это невозможно, потому что есть соглашение о неразглашении. Но у нас есть уникальный проект для заказчика с открытым исходным кодом — это сайт Челябинского зоопарка. Это публичный референс наших инженерных практик. Все репозитории проекта открыты — можно посмотреть примеры реализации разработки по TDD.



Кейс сайта зоопарка

На этом проекте у нас появился новый вопрос: как делать вёрстку по TDD?

Ответ оказался прост: в тестовую стратегию добавили скриншотное тестирование. Итак, что изменилось для этого кейса?

1

Сменили инструмент тестирования.

Заменяли Cypress на Playwright. Подробнее о причинах смены инструмента и вообще о пиксель-перфектном тестировании рассказали наши ребята на UWDC 2025. Но инструмент здесь не главное: на другом, более раннем проекте, мы остались на Cypress и добавили скриншотные тесты на нём.

2

Вёрстка делается по TDD.

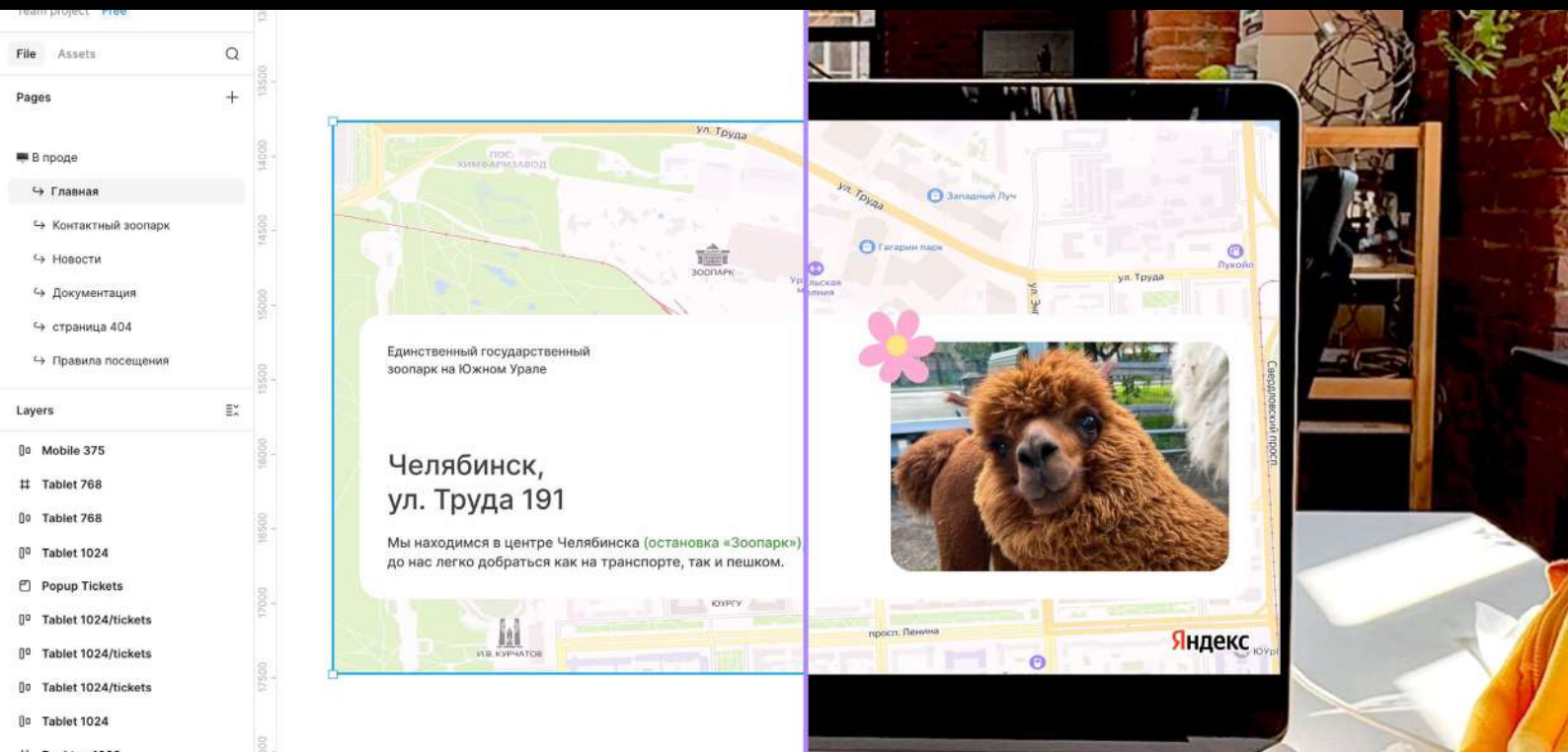
Раньше дизайн и пиксель-пёрфект оставались зоной ручной проверки: открытую Figma сравнивали глазами с открытым сайтом. На зоопарке мы добавили ещё один элемент в тестовую стратегию — скриншотные тесты. Сначала экспортируется изображение из Figma, потом верстается эталон по макету. При запуске автотеста видны расхождения между макетом и вёрсткой: расходящиеся пиксели подсвечиваются красным, и подсчитывается количество промахов.



Запись доклада



Екатерина и Артём, фронтенд-разработчики Tourmaline Core. Запись доклада.



Изображение из кейса сайта Челябинского зоопарка.

3

Вовлекли дизайнеров в TDD-процесс.

Чтобы скриншотные тесты работали, дизайн должен экспортироваться предсказуемо: компонент со всеми слоями, объединение во фреймы, никакой прозрачности в фоне, согласованные размеры и скругления. Поэтому на проекте появился чек-лист дизайнера для передачи макетов в TDD-разработку.

После этой ступени эволюции TDD на фронтенде получили дополнительную возможность сказать: «Наши фронтенды сверстаны пиксель-в-пиксель».

Скрытый бонус: дизайнеры могут не так вьедливо проводить приемку результата, потому что тесты помогают сделать большую часть этой кропотливой работы.

В этом кейсе есть и другие тесты, которые проверяют доступность и интеграцию фронтенда с CMS — подробнее про тестовую стратегию проекта можно посмотреть в докладе.



Какие еще есть публичные кейсы нашей работы по TDD?



Наш сайт

После редизайна мы сделали его на той же основе, что и сайт зоопарка. Поэтому можем подтвердить — рельсы переиспользуемы.



Inner Circle

Платформа для автоматизации бизнес-процессов с открытым исходным кодом. Также используем TDD на сложных системах.



Эпоха ИИ — требования к качеству как никогда высоки

Biz

ИИ стремительно ворвался в нашу жизнь, ускорив написание кода. И именно в этот момент самое важное — это проверка его качества.

TDD здесь оказывается в особенно удобной позиции. Тест, написанный до кода, — это по своей сути формулировка задачи: «вот вход, вот ожидаемый выход, вот граничные случаи».

Цикл TDD в работе с ИИ выглядит так: инженер пишет тест и фиксирует поведение, модель генерирует реализацию, тест проверяет результат. Если тест не проходит — модель видит конкретное расхождение между ожиданием и реальностью. При этом весь накопленный капитал из автотестов гарантирует, что другие части системы не были поломаны.

Это ли не ренессанс Test-Driven Development?

Появление ИИ послужило катализатором улучшения структуры и архитектуры: сейчас как никогда важны спецификации, контракты, user story, итеративная разработка и хорошо написанные автотесты, которые смогут гарантировать качество создаваемого продукта. То, что годами считалось накладными расходами, теперь становится способами сделать ИИ еще более полезным и эффективным.

С приходом ИИ появился новый аргумент в пользу TDD: разработка через тестирование — это способ получить от ИИ предсказуемый код вместо правдоподобного. Ведь быстрая кодогенерация — это в том числе и быстрое создание техдолга. А технический долг — это, как известно, проблема бизнеса.

Команда, у которой есть тестовая стратегия и привычка работать по TDD, использует ИИ с большей отдачей, чем команда, которая работает по другому.

Про техдолг



Как начать работать по TDD?

Tech

Это врезка для тимлидов и руководителей направлений.

Мы знаем, как нелегко переводить команду на Test-Driven Development после многих лет работы с другими привычками. Но кажется, что сейчас самое время для того, чтобы попробовать.

Начать можно с малого: сначала писать тесты на найденный баг. Нашли баг, зафиксировали его автотестом и только после этого починили. Через месяц такой работы у команды есть набор регрессионных тестов и привычка начинать работу с теста. Это будет легким входом.

И пусть вам помогает Мартин Фаулер и его цитата:

«Никакой баг не считается исправленным без автоматического регрессионного теста. В культуре, где есть тесты, когда баг находят, естественная реакция — написать тест, который его обнажает, а потом починить код».

И еще Кент Бек в помощь:

«TDD убирает страх из разработки. А тесты дают разработчику смелость менять, удалять, добавлять».

Можно также написать нам и поболтать про TDD: будем рады поделиться опытом.

Карта наших открытых материалов по теме

Tech

Biz

В этом номере мы разобрали TDD на фрон-тенде. По тому же принципу мы работаем во всех направлениях разработки, только с разной инфраструктурой и разными инструментами.

Если хочется разобраться глубже — вот карта наших открытых материалов по теме.

Если зацепил кейс Челябинского зоопарка



Полный кейс проекта



Подкаст с руководителем команды — про TDD, open source, лицензии, организацию работы

Если интересна разработка по TDD на фронтенде



Воркшоп с TechTrain 2024 — разработка логики по TDD на примере ToDo приложения



Доклад с UWDC 2025 — пиксель-пёрфект тестирование на Playwright



Доклад на Heisenbug 2025 — тестовая стратегия и автоматизация проверки качества на проекте сайта

Если работаете с устройствами и встраиваемой разработкой



Доклад на Heisenbug 2024 — про разработку для устройств и встраивание TDD в проекты с железом



Если интересуют DevOps и автоматизация контроля качества

Доклад на DevOps — про развёртывание и пайплайны, которые можно переиспользовать на проектах разного масштаба

Если работаете с .NET и backend



[В разработке] Плейлист на YouTube — мини-курс по подготовке инфраструктуры и реализации на .NET по TDD





Поделиться мыслями
или оставить отзыв



Узнать больше
о компании